



Lab Mid - A

CSE 4108

Structured Programming I Lab

Table of Contents

Table of Contents.....	1
Submission Instructions.....	1
Task 1: A Small Virtual Machine.....	2
Instruction Set.....	2
Overflow Rules.....	3
Division Rules.....	3
Program Output.....	3
Sample Input and Output.....	3
Task 2: Coprime Hypothesis.....	4
Sample Input and Output.....	4
Task 3: Digit-Shift Cipher.....	4
Sample Input and Output.....	4
Task 4: Cuisine Bear.....	5
Sample Input and Output.....	5

Submission Instructions

Submit individual C files for each of your tasks in the Google Classroom Assignment. The files should be named in the format:

<ID>_Task<TaskNumber>.c

For instance, if your ID is 123456, for task 3, the file name should be 123456_Task3.c. Do not submit executable files.

Task 1: A Small Virtual Machine

You are asked to simulate a very small computer that executes a sequence of instructions. This computer has six registers (containers for holding values, consider them analogues to variables in C):

- A, B, C, D: General-purpose registers
- Z: Always stores the result of the *most recent arithmetic operation*
- S: Status register
 - S = 1 if an overflow occurs during any arithmetic operation
 - S = 0 otherwise

All registers store values of type `long long int`. At the start of the program:

- Registers A, B, C, D, Z are initialized to 0
- Register S is initialized to 0

Instruction Set

Each instruction is given on a separate line. The program must read and execute instructions until the instruction E is encountered.

Instruction Format	Example	Description
L <R> <V>	L A 100	Load value V into register R
C <R1> <R2>	C A D	Copy the value from register R2 into register R1
+ <R1> <R2>	+ B D	Add values of R1 and R2, store result in Z
- <R1> <R2>	- A C	Subtract R2 from R1, store result in Z
* <R1> <R2>	* A B	Multiply R1 and R2, store result in Z
/ <R1> <R2>	/ Z A	Divide R1 by R2 (integer division), store result in Z
^ <R1> <R2>	^ A B	Compute R1 raised to the power of R2, store result in Z
E	E	End execution

Overflow Rules

If any arithmetic operation (+ - * / ^) produces a result that cannot be stored in a `long long int`, set: S = 1. Once set to 1, S remains 1 until the program ends. If no overflow ever occurs, S remains 0.

Division Rules

Division is integer division. **Division by zero will not appear in input.**

Program Output

After the E instruction is processed, print the final values of all registers in the following order, each value should be printed separated by a single space.

A B C D Z S

Sample Input and Output

Sample Input	Sample Output
L A 10 L B 3 + A B C C Z L D 6 - Z D * Z C E	10 3 13 6 91 0
L A 1000000 L B 1000000 * A B ^ Z B E	1000000 1000000 0 0 <undefined> 1

The <undefined> value can be anything as overflow occurs. So any value is acceptable there.

Task 2: Coprime Hypothesis

Suppose your friend came up with the following hypothesis: “There exists a positive integer n such that for every positive integer m , the numbers $nm + 5$ and $n + m$ are coprime.” Coprime numbers (or relatively prime numbers) are a pair or set of integers that share no common factor other than 1. Your friend asks you to help test her hypothesis. For now, you decided to test whether the hypothesis works for any integer from 0 to 100,000. Your friend will give you any number n within this range, and you will try to find the smallest possible positive integer m for which $nm + 5$ and $n + m$ are not coprime. If such a number cannot be found, then that means his hypothesis is correct.

Write a C program that will try to find counterexamples to disprove the hypothesis. The input will be a single integer n , and the output will be the smallest value for which the numbers are not coprime. If they are coprime (for the given range), then just print COPRIME.

Sample Input and Output

Sample Input	Sample Output
2	COPRIME
4	7
24	547

Task 3: Digit-Shift Cipher

Your nephew invented a new simple cipher to encode and decode numbers and asked you to help implement it. The cipher works on positive integers by shifting each digit individually in a modular way. To encrypt a number, each digit is replaced by $(d + 3) \bmod 10$. To decrypt a number, each digit is replaced by $(d + 7) \bmod 10$, effectively reversing the encryption.

Write a C program that would first read a single line of input starting with a character E or D, indicating whether to encrypt or decrypt respectively. After a space, the rest of the line contains the number to be processed. The program must then apply the corresponding digit-wise transformation and output the resulting number as a sequence of digits. The input will not be greater than 9,999,999,999,999.

Sample Input and Output

Sample Input	Sample Output
E 000999000888	333222333111
D 333222333111	000999000888
D 01234	78901

Task 4: Cuisine Bear

Cuisine Bear is a famous food delivery platform, making food accessible to any door nation-wide, albeit with a small delivery charge, depending on the distance from the user's location to the serving store. Weirdly enough (because strings are not covered in the class yet), stores in Cuisine Bear are named with a single letter. Now, stores that are within 100 units (Very Nearby Stores) of the user have a delivery charge of only 0.5 ABD (AB Dollars, a currency unit I made up). Any store that is situated further than 100 units but within 200 units (Nearby Stores) has a delivery charge of 1.5 ABD. Any place after that but within 400 units (Distant Stores) has a charge of 3.5ABD. Any place beyond that is not accessible from the user's location (Inaccessible Stores).

Now, write a C program, that would first, take two numbers, user's location's X and Y coordinates as input. Then it will take N ($1 \leq N \leq 26$), the total number of stores as input, followed by the name (single letter) and X and Y coordinates of the respective store separated by space in separate lines as input. Then the program will output how many Very Nearby, Nearby, Distance, and Inaccessible stores are there from the user' location, followed by the average delivery charge (inaccessible stores are not taken into account while calculating average charge). It will also print out the nearest and furthest locations that can be ordered from. If there are multiple nearest or furthest locations, pick the one appearing first in the input.

[Hint] You can calculate the distance between two points $P_1 = (x_1, y_1)$ and $P_2 = (x_2, y_2)$ using the equation:

$$\text{Distance}(P_1, P_2) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

Sample Input and Output

Sample Input	Sample Output
0 0 4 A 30 40 B 120 90 C 200 300 D 500 500	Very Nearby Stores: 1 Nearby Stores: 1 Distant Stores: 1 Inaccessible Stores: 1 Average Delivery Charge: 1.83 ABD Nearest Store: A Furthest Store: C
125.5 80.25 26 N 50.5 180.0 O 600.0 600.0 P 115.5 70.25 Q 125.5 180.25 R 225.5 80.25 S 400.0 400.0	Very Nearby Stores: 11 Nearby Stores: 7 Distant Stores: 4 Inaccessible Stores: 4 Average Delivery Charge: 1.36 ABD Nearest Store: E Furthest Store: X

T	100.0	100.0
U	200.0	200.0
V	10.0	10.0
W	325.5	80.25
X	125.5	480.25
Y	135.5	90.25
Z	125.5	81.25
A	130.2	85.0
B	200.5	150.75
C	450.0	320.0
D	10.5	20.25
E	125.5	80.25
F	500.2	10.5
G	80.0	60.0
H	250.0	250.0
I	125.5	500.0
J	140.0	90.0
K	300.2	80.25
L	25.0	400.0
M	120.0	75.0